InTrainin — Claude Code Implementation Guide

Version 1.0 | April 2026 | For Engineering Use Only

> This document is the single source of truth for building InTrainin using Claude Code. It is structured as a sequence of self-contained implementation prompts — one per domain — each designed to be pasted directly into a Claude Code session. Read the Architecture Overview (Section 1) in full before starting any prompt. Each prompt builds on the output of the previous one.

How to Use This Guide

Each section contains:

- **Context** — what this domain covers and why it matters
- **Pre-conditions** — what must exist before running this prompt
- **The Prompt** — copy-paste directly into Claude Code
- **Acceptance Criteria** — what "done" looks like before moving to the next prompt
- **Watch-outs** — common failure modes to check for

Run prompts in the exact order presented. Do not skip. If a prompt produces output that fails acceptance criteria, fix it before proceeding.

Section 1 — Architecture Overview (Read First)

Stack

Layer	Technology	Notes
Frontend (PWA)	Next.js 14 (App Router)	Responsive PWA. Learner-facing + Business Admin Panel
Backend API	Node.js + Hono	Domain-structured REST API
Database	PostgreSQL via Supabase	Row-level security. Supabase Auth for session management
CMS	Sanity.io	All role/curriculum content. Non-dev editable
Payments	Paystack	Webhooks for payment confirmation — never trust client
File Storage	Cloudinary	Certificates (PNG), audio cache

Layer	Technology	Notes
Push Notifications	Firebase Cloud Messaging (FCM)	Via web push for PWA
SMS	Termii	OTP delivery, job alerts
TTS	Google Cloud TTS	English; cached per content block
Email	Resend + React Email	Transactional only
Analytics	PostHog	Self-hosted or cloud
Monitoring	Sentry	Error tracking

Repository Structure

```
intrainin/
├── apps/
│   ├── web/          # Next.js PWA (learner + business)
│   └── api/          # Hono API server
├── packages/
│   ├── db/           # Supabase schema, migrations, seed
│   ├── cms/          # Sanity schema definitions
│   ├── shared/       # Types, utils, constants shared across apps
│   └── emails/       # React Email templates
├── .env.example
├── turbo.json
└── package.json      # Turborepo root
```

Domain Map

The API is structured around 7 domains. Each domain maps to a folder under `apps/api/src/domains/`:

1. `auth` — phone OTP + email login, session management
2. `learning` — roles, modules, topics, progress tracking
3. `assessment` — tests, attempts, scoring
4. `certificates` — generation, storage, public verification
5. `jobhub` — worker profiles, hire requests, matching
6. `business` — enterprise accounts, team management
7. `notifications` — push, SMS, email dispatch

Environment Variables (`.env.example`)

```bash
```

```
# Supabase
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Sanity
SANITY_PROJECT_ID=
SANITY_DATASET=production
SANITY_API_TOKEN=

# Paystack
PAYSTACK_SECRET_KEY=
PAYSTACK_WEBHOOK_SECRET=

# Termii
TERMII_API_KEY=
TERMII_SENDER_ID=InTrainin

# Firebase (FCM)
FIREBASE_PROJECT_ID=
FIREBASE_CLIENT_EMAIL=
FIREBASE_PRIVATE_KEY=

# Google Cloud TTS
GOOGLE_TTS_API_KEY=

# Cloudinary
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# Resend
RESEND_API_KEY=

# PostHog
POSTHOG_API_KEY=
POSTHOG_HOST=

# App
NEXT_PUBLIC_APP_URL=
API_URL=
JWT_SECRET=
JWT_REFRESH_SECRET=
```

Prompt 0 — Project Scaffold

Context

Sets up the monorepo, installs all dependencies, and creates the base folder structure. Nothing else should be built until this passes.

Pre-conditions

- Node.js 20+ installed
- pnpm installed globally
- Supabase project created (note URL and keys)
- Sanity project created (note project ID)

The Prompt

```
Create a Turborepo monorepo called `intrainin` with the following structure:

apps/web     - Next.js 14 app using App Router. TypeScript. Tailwind CSS.
shadcn/ui.
apps/api     - Hono API server. TypeScript. Runs on Node.js.
packages/db           - Supabase client + schema migrations using supabase-js and
raw SQL files
packages/cms          - Sanity v3 schema definitions (TypeScript)
packages/shared       - Shared TypeScript types, Zod schemas, and utility
functions
packages/emails       - React Email templates

Root-level setup:
- turbo.json with build, dev, test, lint pipelines
- .env.example with all environment variables listed in the architecture doc
- Root package.json with pnpm workspaces
- ESLint + Prettier configured at root, extended in each package
- TypeScript base config at root, extended per package

In apps/web:
- Configure Next.js with App Router
- Install and initialise shadcn/ui (use "new-york" style, zinc base colour)
- Install Tailwind CSS
- Create folder structure:
  app/
    (auth)/         - login, signup pages (grouped, no shared layout)
    (learner)/      - all learner-facing pages
    (business)/     - all business admin pages
    verify/[code]/  - public certificate verification (no auth)
  components/
```

```
    ui/             - shadcn primitives (auto-generated)
    shared/         - shared composite components
    learner/        - learner-specific components
    business/       - business-specific components
  lib/
    api.ts          - typed API client (fetch wrapper)
    auth.ts         - session helpers
    analytics.ts    - PostHog wrapper

In apps/api:
- Hono app with TypeScript
- Folder structure:
  src/
    index.ts              - entry point, mounts all domain routers
    middleware/
      auth.ts             - JWT verification middleware
      rateLimit.ts        - rate limiting
      webhookVerify.ts    - Paystack HMAC verification
    domains/
      auth/
      learning/
      assessment/
      certificates/
      jobhub/
      business/
      notifications/
    lib/
      supabase.ts         - Supabase admin client
      paystack.ts         - Paystack client
      termii.ts           - Termii SMS client
      cloudinary.ts       - Cloudinary client
      fcm.ts              - Firebase Admin SDK
      tts.ts              - Google Cloud TTS client

In packages/shared, define TypeScript interfaces for all core data models:
- User, Role, Category, Module, Topic, Test, TestQuestion
- Enrollment, TopicProgress, TestAttempt
- Certificate
- JobHubProfile, HireRequest, JobMatch
- Business, BusinessMember

Use Zod for all request/response validation schemas. Export both the Zod schemas
and their inferred TypeScript types.

Do not implement any business logic yet. Scaffold only. All route handlers should
return `{ status: "ok", message: "not implemented" }` as placeholders.
```

```
After scaffolding, verify:
- `pnpm dev` starts both web and api without errors
- TypeScript compiles cleanly across all packages
- No circular dependencies between packages
```

Acceptance Criteria

- `pnpm dev` runs with 0 errors
- `pnpm build` compiles cleanly
- All placeholder routes return 200 `{ status: "ok" }`
- Shared types are importable from both `web` and `api`

Prompt 1 — Database Schema & Migrations

Context

The full Postgres schema for InTrainin. This is the foundation every other domain depends on. Get this right before writing a single line of API logic.

Pre-conditions

- Prompt 0 complete
- Supabase project created and `packages/db` scaffold exists

The Prompt

```
In packages/db, create the full PostgreSQL schema for InTrainin as Supabase
migration files.

Create the following tables. All tables use UUID primary keys generated by
gen_random_uuid(). All tables have created_at TIMESTAMPTZ NOT NULL DEFAULT NOW().
Soft deletes where indicated use deleted_at TIMESTAMPTZ.

--- USERS & AUTH ---

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone VARCHAR(20) UNIQUE,
  email VARCHAR(255) UNIQUE,
  full_name VARCHAR(255) NOT NULL,
  location_city VARCHAR(100),
  location_state VARCHAR(100),
  career_goal_role_id UUID REFERENCES roles(id),
```

```
  account_type VARCHAR(20) NOT NULL DEFAULT 'learner' CHECK (account_type IN
('learner', 'business', 'admin')),
  avatar_url TEXT,
  xp_total INTEGER NOT NULL DEFAULT 0,
  streak_current INTEGER NOT NULL DEFAULT 0,
  streak_last_activity_date DATE,
  fcm_token TEXT,
  notification_prefs JSONB NOT NULL DEFAULT '{"push": true, "sms": true, "email":
true}',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ
);

--- ROLES & CONTENT ---

CREATE TABLE categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  slug VARCHAR(100) NOT NULL UNIQUE,
  icon_name VARCHAR(50),
  display_order INTEGER NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE roles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category_id UUID NOT NULL REFERENCES categories(id),
  title VARCHAR(150) NOT NULL,
  slug VARCHAR(150) NOT NULL UNIQUE,
  description TEXT,
  price_ngn INTEGER NOT NULL DEFAULT 0,
  estimated_hours DECIMAL(4,1),
  is_published BOOLEAN NOT NULL DEFAULT false,
  free_preview_module_count INTEGER NOT NULL DEFAULT 1,
  phase INTEGER NOT NULL DEFAULT 1,
  sanity_id VARCHAR(100),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE modules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  role_id UUID NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  order_index INTEGER NOT NULL,
  is_published BOOLEAN NOT NULL DEFAULT false,
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE topics (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  content_type VARCHAR(30) NOT NULL CHECK (content_type IN ('text', 'guide',
'case_study', 'workflow')),
  content_body JSONB,
  sanity_id VARCHAR(100),
  order_index INTEGER NOT NULL,
  estimated_minutes INTEGER NOT NULL DEFAULT 5,
  is_published BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE tests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module_id UUID REFERENCES modules(id) ON DELETE CASCADE,
  role_id UUID REFERENCES roles(id) ON DELETE CASCADE,
  test_type VARCHAR(20) NOT NULL CHECK (test_type IN ('module', 'final')),
  title VARCHAR(200) NOT NULL,
  questions JSONB NOT NULL DEFAULT '[]',
  pass_mark_pct INTEGER NOT NULL DEFAULT 70,
  time_limit_minutes INTEGER,
  cooldown_hours INTEGER NOT NULL DEFAULT 24,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT test_belongs_to_one CHECK (
    (module_id IS NOT NULL AND role_id IS NULL AND test_type = 'module') OR
    (role_id IS NOT NULL AND module_id IS NULL AND test_type = 'final')
  )
);

--- ROLE PROGRESSION PATHS ---

CREATE TABLE role_progressions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_role_id UUID NOT NULL REFERENCES roles(id),
  to_role_id UUID NOT NULL REFERENCES roles(id),
  progression_type VARCHAR(20) NOT NULL CHECK (progression_type IN ('next',
'adjacent')),
  display_order INTEGER NOT NULL DEFAULT 0,
  UNIQUE(from_role_id, to_role_id)
);

--- ENROLMENTS & PROGRESS ---
```

```
CREATE TABLE enrollments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  role_id UUID NOT NULL REFERENCES roles(id),
  status VARCHAR(20) NOT NULL DEFAULT 'active' CHECK (status IN ('active',
'completed', 'paused')),
  payment_reference VARCHAR(100),
  payment_type VARCHAR(20) CHECK (payment_type IN ('individual', 'enterprise',
'free_trial')),
  enrolled_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at TIMESTAMPTZ,
  UNIQUE(user_id, role_id)
);

CREATE TABLE topic_progress (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  topic_id UUID NOT NULL REFERENCES topics(id),
  status VARCHAR(20) NOT NULL DEFAULT 'not_started' CHECK (status IN
('not_started', 'in_progress', 'completed')),
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  time_spent_seconds INTEGER NOT NULL DEFAULT 0,
  UNIQUE(user_id, topic_id)
);

CREATE TABLE test_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  test_id UUID NOT NULL REFERENCES tests(id),
  score_pct INTEGER NOT NULL,
  passed BOOLEAN NOT NULL,
  attempt_number INTEGER NOT NULL DEFAULT 1,
  answers JSONB NOT NULL DEFAULT '[]',
  taken_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

--- GAMIFICATION ---

CREATE TABLE badges (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug VARCHAR(100) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  icon_url TEXT,
  trigger_type VARCHAR(50) NOT NULL,
```

```
  trigger_value JSONB
);

CREATE TABLE user_badges (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  badge_id UUID NOT NULL REFERENCES badges(id),
  earned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(user_id, badge_id)
);

--- CERTIFICATES ---

CREATE TABLE certificates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  role_id UUID NOT NULL REFERENCES roles(id),
  enrollment_id UUID NOT NULL REFERENCES enrollments(id),
  verification_code UUID NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  issued_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  image_url TEXT,
  is_revoked BOOLEAN NOT NULL DEFAULT false
);

--- JOB HUB ---

CREATE TABLE job_hub_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id),
  is_subscribed BOOLEAN NOT NULL DEFAULT false,
  subscription_plan VARCHAR(20),
  subscription_starts_at TIMESTAMPTZ,
  subscription_expires_at TIMESTAMPTZ,
  preferred_roles UUID[] NOT NULL DEFAULT '{}',
  location_city VARCHAR(100),
  location_state VARCHAR(100),
  availability VARCHAR(30) CHECK (availability IN ('immediate', 'two_weeks',
'one_month')),
  employment_type_pref VARCHAR(30) CHECK (employment_type_pref IN ('full_time',
'part_time', 'contract', 'any')),
  is_visible BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE hire_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  business_id UUID NOT NULL REFERENCES businesses(id),
  role_id UUID NOT NULL REFERENCES roles(id),
  location_city VARCHAR(100) NOT NULL,
  location_state VARCHAR(100),
  positions_count INTEGER NOT NULL DEFAULT 1,
  pay_min INTEGER,
  pay_max INTEGER,
  start_date DATE,
  requirements TEXT,
  certification_required BOOLEAN NOT NULL DEFAULT false,
  status VARCHAR(20) NOT NULL DEFAULT 'open' CHECK (status IN ('open', 'filled',
'closed', 'draft')),
  posted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  expires_at TIMESTAMPTZ,
  payment_reference VARCHAR(100)
);

CREATE TABLE job_matches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  hire_request_id UUID NOT NULL REFERENCES hire_requests(id),
  user_id UUID NOT NULL REFERENCES users(id),
  match_score DECIMAL(5,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending',
'accepted', 'declined', 'shortlisted', 'hired', 'rejected')),
  worker_notified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE(hire_request_id, user_id)
);

--- BUSINESSES ---

CREATE TABLE businesses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_user_id UUID NOT NULL REFERENCES users(id),
  name VARCHAR(200) NOT NULL,
  category VARCHAR(100),
  size_range VARCHAR(30),
  location_city VARCHAR(100),
  location_state VARCHAR(100),
  subscription_plan VARCHAR(20) CHECK (subscription_plan IN ('starter', 'growth',
'business', 'enterprise_plus')),
  subscription_starts_at TIMESTAMPTZ,
  subscription_expires_at TIMESTAMPTZ,
  seat_limit INTEGER NOT NULL DEFAULT 5,
  payment_reference VARCHAR(100),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE business_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  business_id UUID NOT NULL REFERENCES businesses(id),
  user_id UUID REFERENCES users(id),
  invited_phone VARCHAR(20),
  invited_email VARCHAR(255),
  assigned_role_id UUID REFERENCES roles(id),
  job_title VARCHAR(100),
  status VARCHAR(20) NOT NULL DEFAULT 'invited' CHECK (status IN ('invited',
'active', 'removed')),
  invited_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  joined_at TIMESTAMPTZ,
  UNIQUE(business_id, user_id)
);

--- NOTIFICATIONS ---

CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  type VARCHAR(50) NOT NULL,
  title VARCHAR(200) NOT NULL,
  body TEXT,
  data JSONB,
  is_read BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

--- INDEXES ---
Add indexes on:
- users(phone), users(email)
- enrollments(user_id), enrollments(role_id)
- topic_progress(user_id, topic_id)
- test_attempts(user_id, test_id)
- certificates(verification_code)
- hire_requests(status, role_id, location_city)
- job_matches(hire_request_id), job_matches(user_id)
- business_members(business_id), business_members(user_id)
- notifications(user_id, is_read)

--- ROW-LEVEL SECURITY ---
Enable RLS on all tables. Apply policies:
- users: SELECT/UPDATE own row only (id = auth.uid())
- enrollments: SELECT/INSERT own rows only
```

```
- topic_progress: SELECT/INSERT/UPDATE own rows only
- test_attempts: SELECT/INSERT own rows only
- certificates: SELECT own rows; public SELECT on verification_code lookup
- job_hub_profiles: SELECT/UPDATE own row
- job_matches: SELECT own rows
- businesses: full access to owner; read access to members
- business_members: SELECT if business owner or self
- notifications: SELECT/UPDATE own rows

After writing migrations, create a seed file (seed.sql) that inserts:
- 7 categories (from the PRD)
- 3 sample roles (Cashier, Waiter/Waitress, Delivery Rider) with 2 modules each, 3
topics per module, and 1 module test per module + 1 final exam
- 5 badges (first_module, seven_day_streak, top_score, multi_role, first_cert)
- Role progressions connecting the 3 seed roles

Document all migration files with comments. Place migrations in
packages/db/migrations/ numbered sequentially (001_init.sql, 002_indexes.sql,
003_rls.sql, 004_seed.sql).
```

Acceptance Criteria

- `supabase db push` applies all migrations cleanly
- Seed data visible in Supabase Table Editor
- RLS policies verified: a query with a test user JWT only returns that user's rows
- No orphaned foreign key references

Watch-outs

- The `career_goal_role_id` foreign key in `users` creates a circular reference with `roles`. Add it as an ALTER TABLE after the roles table is created, not inline.
- RLS `auth.uid()` requires Supabase Auth — if testing with service role key, RLS is bypassed. Test RLS explicitly with anon key.

Prompt 2 — Authentication Domain

Context

Phone OTP is the primary auth method for learners. Email+password is for business accounts. JWT + refresh token session management. This gate controls access to all downstream features.

Pre-conditions

- Prompt 1 complete

- Termii account with API key
- Supabase Auth configured

The Prompt

```
Implement the full authentication domain for InTrainin in
apps/api/src/domains/auth/.

REQUIREMENTS:

Phone OTP Flow (primary — for learner accounts):
1. POST /auth/otp/request
   - Body: { phone: string } (E.164 format, e.g. "+2348012345678")
   - Validate phone format with Zod
   - Generate a 6-digit OTP, store it in Supabase with a 10-minute expiry (use a
separate `otp_codes` table: phone, code, expires_at, used_at)
   - Send OTP via Termii SMS to the phone number
   - Message template: "Your InTrainin verification code is {CODE}. Valid for 10
minutes. Do not share this code."
   - Return: { success: true, expires_in: 600 }
   - Rate limit: max 3 OTP requests per phone per hour

2. POST /auth/otp/verify
   - Body: { phone: string, code: string }
   - Validate OTP: must exist, not expired, not used
   - Mark OTP as used
   - If user with this phone doesn't exist: create user record (account_type =
'learner')
   - If user exists: retrieve user
   - Generate JWT access token (15min expiry) and refresh token (30 days expiry)
   - Store refresh token hash in a `refresh_tokens` table (user_id, token_hash,
expires_at, created_at)
   - Return: { access_token, refresh_token, user: { id, phone, full_name,
account_type } }

Email+Password Flow (for business accounts):
3. POST /auth/email/register
   - Body: { email, password, full_name, business_name }
   - Validate email, password (min 8 chars, 1 number), full_name
   - Check email not already registered
   - Hash password with bcrypt (12 rounds)
   - Create user record (account_type = 'business')
   - Create business record (owner_user_id = new user id, name = business_name)
   - Send welcome email via Resend
   - Return: { success: true, message: "Check your email to continue" }
```

```
4. POST /auth/email/login
   - Body: { email, password }
   - Find user by email; verify password hash
   - Generate JWT + refresh token
   - Return same shape as OTP verify

Session Management:
5. POST /auth/refresh
   - Body: { refresh_token: string }
   - Validate refresh token: hash lookup, not expired
   - Issue new access token (and optionally rotate refresh token)
   - Return: { access_token, refresh_token? }

6. POST /auth/logout
   - Authenticated route
   - Invalidate current refresh token (set expires_at to NOW())
   - Return: { success: true }

Profile Setup (called after first login if profile incomplete):
7. PATCH /auth/profile
   - Authenticated route
   - Body: { full_name?, location_city?, location_state?, career_goal_role_id? }
   - Update user record
   - Return updated user object

JWT Middleware (apps/api/src/middleware/auth.ts):
- Extract Bearer token from Authorization header
- Verify JWT signature using JWT_SECRET
- Attach decoded user to context: c.set('user', decodedUser)
- Return 401 if missing or invalid
- Skip for public routes: /auth/*, /roles (GET), /verify/*

Security requirements:
- Never log OTP codes or passwords
- OTP codes must be constant-time compared (use crypto.timingSafeEqual)
- JWT payload: { sub: userId, type: account_type, iat, exp }
- Refresh tokens stored as SHA-256 hash only — never raw
- Add `otp_codes` table migration (phone VARCHAR, code VARCHAR(6), expires_at
TIMESTAMPTZ, used_at TIMESTAMPTZ, created_at TIMESTAMPTZ)
- Add `refresh_tokens` table migration

Write Zod schemas for all request bodies in packages/shared/src/schemas/auth.ts.
Write TypeScript types for JWT payload in packages/shared/src/types/auth.ts.
Export a typed `useAuth` context hook for the web app in apps/web/lib/auth.ts that
reads from localStorage (access token) and handles refresh automatically.
```

Acceptance Criteria

- Phone OTP request creates an OTP_codes record and triggers Termii call (mock in test)
- OTP verify returns a valid JWT that passes middleware verification
- JWT middleware correctly blocks requests without token
- Refresh token rotation works end-to-end
- Business registration creates both user and business records atomically

Watch-outs

- Termii's API has different endpoint structures for `device_id` vs bulk. Use the single-send endpoint for OTP.
- `crypto.timingSafeEqual` requires both buffers to be the same length — pad if necessary.

Prompt 3 — Learning Domain (API)

Context

The core of the product. Serves curriculum content, tracks learner progress, enforces the linear progression rules (must complete topics in order, tests unlock only after all topics done), and awards XP.

Pre-conditions

- Prompt 2 complete
- Sanity project created with `SANITY_PROJECT_ID` set

The Prompt

```
Implement the full learning domain in apps/api/src/domains/learning/.

CONTENT ENDPOINTS (public or lightly authenticated):

1. GET /roles
   - Public (no auth required)
   - Returns all published roles grouped by category
   - Each role: id, title, slug, category, price_ngn, estimated_hours,
module_count, is_free_preview_available
   - Cache response for 15 minutes (use Hono's cache middleware or a simple in-
memory TTL cache)

2. GET /roles/:slug
   - Public
```

```
   - Returns full role detail: all published modules, each module's topics (title,
content_type, estimated_minutes, order_index — no content_body)
   - Also returns: whether current authenticated user (if any) is enrolled, their
progress percentage

3. GET /roles/:slug/enroll-preview
   - Public
   - Returns Module 1 content in full (free preview)

ENROLLMENT ENDPOINTS (authenticated):

4. POST /enrollments
   - Auth required
   - Body: { role_id: string, payment_reference?: string }
   - Validate:
     a. Role exists and is published
     b. User not already enrolled
     c. If role has price > 0: payment_reference must exist and be verified (query
Paystack verify endpoint)
     d. If user is a business_member with this role assigned: allow without
payment (enterprise)
   - Create enrollment record (status: active)
   - If payment verified: log payment_reference, set payment_type = 'individual'
   - Return enrollment object

5. GET /enrollments/me
   - Auth required
   - Returns all of current user's enrollments with role details, progress
percentage, and certificate status

PROGRESS ENDPOINTS (authenticated):

6. GET /learning/:enrollment_id/progress
   - Auth required; must own enrollment
   - Returns full progress breakdown: each module with completion status, each
topic with completion status
   - Computes and returns: overall_pct, current_module_id, next_topic_id

7. POST /learning/topics/:topic_id/complete
   - Auth required
   - Validates:
     a. User is enrolled in the role this topic belongs to
     b. All previous topics in the module are completed (linear progression
enforcement)
   - Creates or updates topic_progress record (status: completed, completed_at:
NOW())
   - Upserts time_spent_seconds from request body { time_spent_seconds: number }
```

```
   - Awards 10 XP to user (UPDATE users SET xp_total = xp_total + 10)
   - Updates user streak: if last activity was yesterday, increment streak; if
today already, no change; if >1 day gap, reset to 1. Update
streak_last_activity_date.
   - Checks if all topics in module are now complete → if so, check if module test
exists → set a flag in response: { module_test_unlocked: true/false }
   - Return: { success: true, xp_awarded: 10, new_xp_total: number, streak:
number, module_test_unlocked: bool }

8. GET /learning/topics/:topic_id/content
   - Auth required
   - Validates user is enrolled and topic is accessible (previous topics complete
OR it's the first topic OR free preview)
   - Fetches content: first check topics.content_body in DB; if null, fetch from
Sanity using topics.sanity_id
   - Returns content_body JSON

SANITY INTEGRATION:
- Implement a Sanity client in apps/api/src/lib/sanity.ts
- Use @sanity/client
- Fetch content by document _id (= topic.sanity_id)
- Cache fetched content in DB (write to topics.content_body) after first fetch
- If Sanity is unavailable, fall back to content_body in DB

TTS ENDPOINT:
9. GET /learning/topics/:topic_id/audio
   - Auth required
   - Validates enrollment and access as above
   - Extracts plain text from content_body JSON (strip markdown/formatting)
   - Checks Cloudinary for cached audio: intrainin/tts/{topic_id}.mp3
   - If cached: return { audio_url: string }
   - If not cached: call Google Cloud TTS API with the text, upload result to
Cloudinary, cache URL, return it
   - Language: 'en-NG' (Nigerian English) as default

Write all Zod request schemas in packages/shared/src/schemas/learning.ts.
Write a progress calculation utility in packages/shared/src/utils/progress.ts that
computes:
- Percentage complete for a module given completed topic IDs
- Percentage complete for a role given completed module IDs + passed module tests
```

Acceptance Criteria

- Role catalogue returns correctly grouped, cached data
- Free preview returns Module 1 content without auth
- Topic completion correctly enforces linear progression (attempt out-of-order → 403)

- XP awarded on topic completion; DB updated
- Streak logic: day 1→day 2 increments, day 1→day 3 resets
- Sanity fallback to DB content_body works if Sanity is unreachable
- TTS endpoint returns cached Cloudinary URL on second call (no re-fetch from Google)

Prompt 4 — Assessment Domain

Context

Module tests and final exams. Controls access (cooldown enforcement), scores submissions, triggers certificate generation on final exam pass.

Pre-conditions

- Prompt 3 complete

The Prompt

```
Implement the full assessment domain in apps/api/src/domains/assessment/.

TEST ACCESS:

1. GET /tests/:test_id
   - Auth required
   - Returns test metadata (title, question count, time_limit_minutes,
pass_mark_pct)
   - Does NOT return correct answers
   - Validates access:
     - For module test: all topics in the module must be completed by this user
     - For final exam: all module tests in the role must be passed by this user
   - Checks cooldown: if user has a failed attempt, last attempt must be >
cooldown_hours ago
   - Returns: { test, is_accessible: bool, cooldown_remaining_minutes: number |
null }

2. GET /tests/:test_id/questions
   - Auth required
   - Same access validation as above
   - Returns questions array WITHOUT correct_answer field
   - Shuffle question order per request (use Fisher-Yates, seeded by
user_id+test_id+attempt_number for reproducibility)
   - Each question: { id, question_text, question_type (mcq|scenario), options:
string[] }
```

```
TEST SUBMISSION:

3. POST /tests/:test_id/submit
   - Auth required
   - Body: { answers: Array<{ question_id: string, selected_option: string }> }
   - Validation:
     - Re-validate test access (idempotency guard)
     - Answers array length must match question count
   - Scoring:
     - Compare each submitted answer to correct_answer in questions JSON
     - Compute score_pct = (correct_count / total_count) * 100, rounded to integer
     - passed = score_pct >= test.pass_mark_pct
   - Persist test_attempt record (attempt_number = previous attempts count + 1)
   - If passed AND test_type = 'module':
     - Award 50 XP
     - Check badge eligibility: "Top Score" if score_pct >= 90
   - If passed AND test_type = 'final':
     - Award 200 XP
     - Update enrollment status to 'completed', set completed_at
     - Trigger certificate generation (call internal certificate service — POST to
/certificates/generate internally)
     - Check badge eligibility: "Top Score", "First Certificate", "Multi-Role
Learner"
   - Return: {
       attempt_id, score_pct, passed, correct_count, total_count,
       xp_awarded: number,
       badges_earned: Badge[],
       certificate_id?: string  (if final exam passed)
     }

BADGE EVALUATION:
Create a badge evaluator function in
apps/api/src/domains/assessment/badgeEvaluator.ts:

function evaluateBadges(userId, context: { test_type, score_pct,
total_certs_count, total_enrollments_count }): Promise<Badge[]>

Checks:
- "first_module" — awarded on first ever module test pass (check user_badges
table)
- "seven_day_streak" — check users.streak_current >= 7
- "top_score" — score_pct >= 90 on this attempt
- "first_cert" — total_certs_count === 1 (just earned first certificate)
- "multi_role" — total_enrollments_count >= 2

For each earned badge: INSERT INTO user_badges (unless already earned — use ON
CONFLICT DO NOTHING).
```

```
Return array of newly awarded badges.

ATTEMPT HISTORY:

4. GET /tests/:test_id/attempts
   - Auth required
   - Returns all attempts by current user for this test
   - Includes: score_pct, passed, taken_at, attempt_number
   - Does NOT return answers

Write all Zod schemas in packages/shared/src/schemas/assessment.ts.
```

Acceptance Criteria

- Module test blocked if topics not complete → 403 with clear message
- Final exam blocked if any module test not passed → 403
- Cooldown enforced: second attempt within cooldown window → 403 with `cooldown_remaining_minutes`
- Scoring is correct: 8/10 correct = 80%
- Final exam pass triggers certificate generation call
- Badges awarded correctly and not duplicated (idempotent)
- Answers not returned in questions response

Prompt 5 — Certificate Domain

Context

Auto-generation on final exam pass. Server-side PNG rendering with learner name, role, date, and verification code. Public verification page.

Pre-conditions

- Prompt 4 complete
- Cloudinary configured

The Prompt

```
Implement the certificate domain in apps/api/src/domains/certificates/ and the
public verification page in apps/web.

CERTIFICATE GENERATION SERVICE (apps/api/src/domains/certificates/generate.ts):
```

```
Create a function: generateCertificate(enrollment_id: string):
Promise<Certificate>

Steps:
1. Fetch enrollment + user + role from DB
2. Check no certificate already exists for this enrollment (idempotent)
3. Generate a UUID verification_code
4. Render certificate as PNG using @napi-rs/canvas (NOT Puppeteer — canvas is
lighter):
   Canvas size: 1200 x 850 px
   Design:
   - Background: white (#FFFFFF)
   - Top banner (0,0 to 1200,120): filled with InTrainin brand blue (#1A56DB)
   - "InTrainin" wordmark: bold, 48px, white, top-left in banner with 40px padding
   - "Certificate of Completion" text: white, 20px, top-right in banner
   - Body: centered vertical layout
   - "This certifies that" — gray, 18px, centered, y=200
   - Learner full name — bold, 52px, #111827, centered, y=260
   - "has successfully completed the" — gray, 18px, centered, y=330
   - Role title — bold, 36px, #1A56DB, centered, y=380
   - "InTrainin Role Certification" — gray, 16px, centered, y=430
   - Thin horizontal rule (#E5E7EB) at y=480, from x=100 to x=1100
   - "Date: {DD Month YYYY}" — gray, 16px, left-aligned at x=100, y=510
   - "Verification ID: {code}" — gray, 14px, left-aligned at x=100, y=540
(truncate UUID to first 8 chars for display)
   - QR code bottom-right: render QR code for URL
https://intrainin.com/verify/{verification_code} using 'qrcode' npm package, place
at x=950,y=480, size 150x150
   - Bottom: full-width InTrainin blue bar (y=800 to 850)

5. Convert canvas to PNG buffer
6. Upload to Cloudinary: folder=intrainin/certificates, public_id=
{verification_code}
7. Insert certificate record into DB
8. Return certificate object with image_url

API ENDPOINTS:

1. POST /certificates/generate (internal — called by assessment domain)
   - Auth required (service-level: validate request comes from internal API only
via a shared internal secret header)
   - Body: { enrollment_id: string }
   - Calls generateCertificate(enrollment_id)
   - Returns certificate object

2. GET /certificates/me
   - Auth required
```

```
   - Returns all certificates for current user: { id, role_title, issued_at,
image_url, verification_code, share_url }
   - share_url = https://intrainin.com/verify/{verification_code}

3. GET /verify/:code (PUBLIC — no auth)
   - Rate limit: 20 requests per minute per IP
   - Look up certificate by verification_code
   - If found and not revoked: return { valid: true, full_name, role_title,
issued_at }
   - If not found or revoked: return { valid: false }
   - Add appropriate cache headers (max-age=3600)

WEB: PUBLIC VERIFICATION PAGE (apps/web/app/verify/[code]/page.tsx):
- Server-side rendered (Next.js RSC)
- Call the API GET /verify/:code
- If valid: show a clean verification page:
  - Green check icon
  - "Certificate Verified" heading
  - Learner name, role, date issued
  - InTrainin branding
  - "This certificate was issued by InTrainin and is authentic."
- If invalid: show red X, "Certificate Not Found or Invalid"
- This page must render without JavaScript (for WhatsApp link previews)
- Set og:title and og:description meta tags for WhatsApp preview

SHARING:
Add a utility in packages/shared/src/utils/share.ts:
- generateWhatsAppShareText(certificate): string — returns pre-composed message
- generateLinkedInShareUrl(certificate): string
```

Acceptance Criteria

- Certificate PNG renders correctly with all fields populated
- QR code in certificate links to correct verify URL
- Verification page loads without JS and shows correct data
- Verification endpoint returns 404-equivalent JSON for invalid codes
- Rate limiting on verification endpoint works
- generateCertificate is idempotent (second call returns existing cert without re-generating)

Prompt 6 — Payment Integration (Paystack)

Context

All revenue flows through Paystack. Individual course purchases, Job Hub subscriptions, and hire request payments. Webhook-first — never trust client-side confirmation.

Pre-conditions

- Prompts 1–5 complete
- Paystack account with live keys

The Prompt

```
Implement the full Paystack payment integration in apps/api/src/domains/payments/
and apps/api/src/lib/paystack.ts.

PAYSTACK CLIENT (apps/api/src/lib/paystack.ts):
Wrap Paystack API calls with typed functions:
- initializeTransaction({ email, amount_kobo, reference, metadata, callback_url
}): Promise<PaystackInitResponse>
- verifyTransaction(reference: string): Promise<PaystackVerifyResponse>
- createSubscription(params): Promise<any>
- Types for all responses

PAYMENT FLOWS TO IMPLEMENT:

1. Individual Course Purchase
   POST /payments/course/initiate
   - Auth required
   - Body: { role_id: string }
   - Validate: role exists, user not already enrolled, role has a price
   - Generate a unique payment reference: IT-COURSE-{userId}-{roleId}-{timestamp}
   - Amount: role.price_ngn * 100 (kobo)
   - Call Paystack initializeTransaction with:
     - metadata: { user_id, role_id, payment_type: 'course' }
     - callback_url: {APP_URL}/payment/callback
   - Store a pending payment record in a `payments` table (reference, user_id,
type, amount_kobo, status: 'pending', metadata)
   - Return: { authorization_url, reference }

2. Job Hub Subscription
   POST /payments/jobhub/initiate
   - Auth required
   - Body: { plan: 'monthly' | 'annual' }
   - Amounts: monthly = ₦1,000 (100000 kobo), annual = ₦8,000 (800000 kobo)
   - Same flow as above; metadata: { user_id, plan, payment_type:
```

```
'jobhub_subscription' }
   - Return: { authorization_url, reference }

3. Hire Request Payment
   POST /payments/hire-request/initiate
   - Auth required (business account only)
   - Body: { hire_request_id: string }
   - Validate: hire_request belongs to this business, status is 'draft'
   - Amount based on positions_count (₦3,000-₦10,000 per request, use a pricing
function)
   - metadata: { business_id, hire_request_id, payment_type: 'hire_request' }
   - Return: { authorization_url, reference }

4. Enterprise Package
   POST /payments/enterprise/initiate
   - Auth required (business account only)
   - Body: { plan: 'starter' | 'growth' | 'business', months: 1 | 3 | 6 | 12 }
   - Pricing:
     - starter: ₦15,000/month
     - growth: ₦40,000/month
     - business: ₦80,000/month
   - Multiply by months, apply discount: 3mo=5% off, 6mo=10% off, 12mo=15% off
   - Return: { authorization_url, reference, total_amount, discount_applied }

WEBHOOK HANDLER:
POST /payments/webhook (PUBLIC — no auth, but HMAC verified)

In middleware (webhookVerify.ts):
- Extract X-Paystack-Signature header
- Compute HMAC-SHA512 of raw request body using PAYSTACK_WEBHOOK_SECRET
- Compare with header value using crypto.timingSafeEqual
- Reject with 400 if mismatch
- IMPORTANT: Read raw body as Buffer before any JSON parsing

In webhook handler, process these events:
- charge.success:
  - Fetch payment record by reference
  - Verify transaction amount matches expected amount (reject if tampered)
  - Update payment status to 'completed'
  - Based on payment_type in metadata:
    - 'course': call enrollments service to create enrollment (payment_type:
'individual')
    - 'jobhub_subscription': activate or extend job_hub_profiles subscription
    - 'hire_request': update hire_request status from 'draft' to 'open'
    - 'enterprise': update business subscription_plan, seat_limit, expiry
  - Send confirmation notification (in-app + email)
```

```
- charge.failed:
  - Update payment record status to 'failed'
  - No further action needed

Add a `payments` table migration:
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  reference VARCHAR(100) NOT NULL UNIQUE,
  user_id UUID NOT NULL REFERENCES users(id),
  business_id UUID REFERENCES businesses(id),
  payment_type VARCHAR(30) NOT NULL,
  amount_kobo INTEGER NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN ('pending',
'completed', 'failed')),
  metadata JSONB NOT NULL DEFAULT '{}',
  paystack_response JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

PAYMENT CALLBACK PAGE (apps/web/app/payment/callback/page.tsx):
- Reads ?reference= from URL
- Calls GET /payments/status/:reference
- Shows loading spinner while polling (poll every 2s, up to 30s)
- On success: redirect to /dashboard with success toast
- On failure: show error with retry CTA

Add GET /payments/status/:reference endpoint:
- Auth required (own payments only)
- Returns { status: 'pending' | 'completed' | 'failed', redirect_to?: string }
```

Acceptance Criteria

- Webhook HMAC validation rejects tampered requests
- Course enrollment created ONLY via webhook, never client-side
- Amount tampering rejected (if Paystack confirms ₦100 but expected ₦2000 → reject + log alert)
- Enterprise subscription correctly calculates discounts
- Callback page polls and redirects correctly
- All payment records persisted before calling Paystack (reference exists before redirect)

Watch-outs

- Hono's `req.text()` vs `req.json()` — read raw body as text/buffer in webhook middleware BEFORE any JSON parsing, or HMAC verification will fail.

- Paystack test mode and live mode use different keys. Use `PAYSTACK_SECRET_KEY` from env — never hardcode.

Prompt 7 — Job Hub Domain

Context

Two-sided matching. Workers subscribe and set preferences. Employers post hire requests. A matching algorithm scores candidates and surfaces the top 10. Notifications go out on new matches.

Pre-conditions

- Prompts 2–6 complete

The Prompt

```
Implement the Job Hub domain in apps/api/src/domains/jobhub/.

WORKER ENDPOINTS:

1. POST /jobhub/profile
   - Auth required
   - Body: { preferred_roles: string[], location_city, location_state,
availability, employment_type_pref }
   - Upsert job_hub_profile for current user
   - Return updated profile

2. GET /jobhub/profile/me
   - Auth required
   - Return current user's job hub profile including subscription status

3. GET /jobhub/matches
   - Auth required; subscription check (is_subscribed = true AND
subscription_expires_at > NOW())
   - Returns job matches for current user, sorted by match_score DESC, most recent
first
   - Include hire request details: role, location, pay_range, employer_name (if
not anonymous), start_date
   - Filter: only matches where hire_request.status = 'open'

4. PATCH /jobhub/matches/:match_id
   - Auth required (own match only)
   - Body: { status: 'accepted' | 'declined' }
   - Update match status
```

```
   - If accepted: notify employer via in-app notification

EMPLOYER ENDPOINTS:

5. POST /jobhub/hire-requests
   - Auth required (business account)
   - Body: { role_id, location_city, location_state, positions_count, pay_min?,
pay_max?, start_date?, requirements?, certification_required }
   - Validate business has active subscription OR will pay per-request
   - Create hire_request with status 'draft' (becomes 'open' after payment
confirms via webhook)
   - Return hire_request + payment_initiation_url if payment required

6. GET /jobhub/hire-requests/me
   - Auth required (business account)
   - Returns all hire requests for this business with match counts per status

7. GET /jobhub/hire-requests/:id/candidates
   - Auth required (must own hire_request)
   - Returns top 10 candidates, sorted by match_score
   - Each candidate: { user_id, first_name (first name only — no full name until
shortlisted), location_city, certificates (role + issued_at), xp_total,
test_score_for_role? }

8. POST /jobhub/hire-requests/:id/shortlist
   - Auth required (must own hire_request)
   - Body: { user_id: string }
   - Update job_match status to 'shortlisted'
   - Reveal full name and contact to employer at this point (return it in
response)
   - Notify worker of shortlisting via push + SMS

MATCHING ALGORITHM (jobhub/matcher.ts):
Create function: runMatchingForHireRequest(hire_request_id: string): Promise<void>

Logic:
1. Fetch hire request (role_id, location_city, certification_required)
2. Query job_hub_profiles WHERE:
   - is_subscribed = true AND subscription_expires_at > NOW()
   - is_visible = true
   - preferred_roles contains hire_request.role_id OR
     preferred_roles contains any adjacent role (from role_progressions)
3. For each candidate, compute match_score (0-100):
   - Role exact match: +40 points
   - Role adjacent match: +20 points
   - Location match (same city): +25 points
   - Location match (same state only): +10 points
```

```
   - Has InTrainin certificate for the role: +25 points
   - Availability = 'immediate': +10 bonus points
   - Cap at 100
4. Filter candidates with score >= 30
5. Sort by score DESC, take top 50
6. Upsert into job_matches (use ON CONFLICT DO UPDATE to refresh score)
7. For new matches (status was null before): set worker_notified_at = NOW(), queue
notification

TRIGGER: Call runMatchingForHireRequest whenever:
- A hire_request status changes from 'draft' to 'open' (after payment)
- A new enrollment is completed (re-run matching for relevant open hire requests)

NOTIFICATIONS (triggered from jobhub):
- Worker new match: push + SMS "New job opportunity for {role} in {city}. Open
InTrainin to view."
- Worker shortlisted: push + SMS "Great news! A business is interested in your
profile for {role}."
```

Acceptance Criteria

- Match algorithm correctly scores exact vs adjacent role
- Location matching: same city scores higher than same state
- Certified candidates score higher than uncertified
- Only subscribed, visible workers appear in candidate pool
- First name only shown until shortlisted — full contact revealed on shortlist action
- Matching runs automatically when hire request activates
- Worker notifications sent on new match

Prompt 8 — Business Admin Panel (API + UI)

Context

B2B dashboard for enterprise subscribers. Manage team members, assign courses, track progress, and access the Job Hub for hiring. This is a separate UI layout from the learner-facing app.

Pre-conditions

- Prompts 2–7 complete

The Prompt

```
Implement the Business Admin Panel — both API endpoints and the Next.js UI.

API (apps/api/src/domains/business/):

1. GET /business/me
   - Auth required (business account)
   - Returns business record + subscription details + seat usage (active members
count vs seat_limit)

2. POST /business/members
   - Auth required (business owner)
   - Body: { members: Array<{ phone?: string, email?: string, job_title?: string,
role_id?: string }> }
   - Validate: seat limit not exceeded
   - For each member:
     - Create business_members record (status: 'invited')
     - If user with phone/email exists: link user_id
     - Send invitation: SMS (if phone) or email (if email only)
     - SMS: "You've been invited to train on InTrainin by {business_name}. Start
your {role_title} course: {link}"
     - Email: render InvitationEmail template from packages/emails
   - Return: { invited_count, failed: [] }

3. POST /business/members/bulk
   - Auth required (business owner)
   - Body: multipart form with CSV file
   - CSV columns: phone, email, job_title, role_slug
   - Parse CSV, validate rows, call same logic as above in batch
   - Return: { total_rows, invited_count, skipped: [], errors: [] }

4. GET /business/members
   - Auth required (business owner)
   - Returns all members with: name (if joined), phone, job_title, assigned_role,
enrollment status, module completion %, certificate status
   - Sortable by: name, completion_pct, joined_at

5. DELETE /business/members/:id
   - Auth required (business owner)
   - Sets business_member status to 'removed'
   - Does NOT delete the user or their learning progress (it's theirs to keep)

6. GET /business/analytics
   - Auth required (business owner)
   - Returns:
```

- overall_completion_rate: avg completion % across all active members
- per_role_stats: [{ role_title, enrolled_count, completed_count, avg_score }]
- at_risk_members: members with 0 activity in last 7 days who haven't completed
- top_performers: members with highest completion % or XP

BUSINESS ADMIN UI (apps/web/app/(business)/):

Create the following pages. Use shadcn/ui components throughout. The business admin layout is distinct from the learner layout — use a sidebar navigation.

Layout: apps/web/app/(business)/layout.tsx
- Left sidebar with navigation: Dashboard, Team, Progress, Hire (Job Hub), Settings, Billing
- Top bar with business name and user avatar
- Sidebar collapses on mobile

Pages to build:

/business/dashboard
- Summary cards: Active Members, Avg Completion Rate, Open Hire Requests, Certificates Earned This Month
- Line chart: team completion over time (use recharts)
- At-risk members list (members inactive 7+ days)

/business/team
- Table of all members: name/phone, role, status (invited/active), progress bar, cert badge
- Add Member button → inline form (single add)
- Bulk Import button → file upload for CSV
- Click row → member detail drawer (shows full progress timeline)
- Filter by: role, status (invited/active/completed)

/business/progress
- Grouped by role: expand to see member-by-member progress
- Export as CSV button (downloads current view)

/business/hire
- Open hire requests list with match counts
- New Hire Request button → multi-step form (role, location, pay range, requirements)
- Candidates view for each open request (name shown only after shortlisting)
- Interview request flow

/business/settings
- Edit business name, category, location

```
- Team role defaults (map job titles to InTrainin roles)

/business/billing
- Current plan, seat usage, renewal date
- Upgrade plan CTA
- Payment history table

All forms use react-hook-form + zod resolver.
All data fetching uses SWR with the typed API client from apps/web/lib/api.ts.
All tables support sorting and are paginated (page size: 20).
```

Acceptance Criteria

- CSV bulk import handles 50-row file without timeout
- Seat limit enforced: adding member beyond limit returns 403 with clear error
- Team table shows accurate real-time progress
- At-risk members correctly identified (no activity in 7 days)
- Progress export generates correct CSV
- Business member removed: their learning progress unaffected, enrollment still works

---

Prompt 9 — Learner-Facing PWA (Web UI)

Context

The primary learner interface. Mobile-first, PWA. Covers: onboarding, role catalogue, learning flow, test taking, dashboard, certificate sharing, Job Hub, and career roadmap.

Pre-conditions

- All API prompts (0–8) complete

The Prompt

```
Build the complete learner-facing PWA in apps/web/app/(learner)/.

Design direction: Clean, confident, mobile-first. Think of a learner on a Tecno
Android phone with a 5-inch screen. Prioritise readability, thumb-friendly tap
targets (min 44px), and fast-feeling interactions. Primary brand colour: #1A56DB.
Use a warm, accessible sans-serif — DM Sans from Google Fonts. The tone is
encouraging and direct — not corporate.

AUTHENTICATION PAGES (apps/web/app/(auth)/):
```

/login
- Phone number input (auto-format Nigerian numbers, +234 prefix)
- "Get OTP" button → shows 6-digit OTP input
- OTP input: 6 separate single-digit inputs, auto-advance on digit entry
- Countdown timer (10 min) with "Resend OTP" after 60s
- Below the phone flow: "Are you a business? Sign in with email →" link

/signup (business)
- Email, password, full name, business name
- Password strength indicator

/onboarding (shown after first login if profile incomplete)
- Step 1: "What's your name?" — single text input
- Step 2: "Where are you?" — city selector (Nigerian cities list)
- Step 3: "What's your goal?" — role category grid (icon cards), then role selection
- Progress dots at top
- Skip allowed after Step 1

LEARNER PAGES:

/dashboard (home after login)
- Greeting: "Good morning, Amara 👋"
- Streak badge (flame icon + number)
- In-progress roles: horizontal scroll of role cards with progress ring
- "Resume" CTA card: shows exact topic to continue
- Recent certificates: small tile row
- Job Hub teaser (if not subscribed): "3 jobs match your skills →"

/roles (role catalogue)
- Search bar (sticky at top)
- Category filter chips (horizontal scroll)
- Role cards in 2-column grid:
  Each card: role icon, title, category, module count, estimated hours, price, "Preview" button
- Skeleton loading states

/roles/[slug] (role detail)
- Hero: role title, category, estimated hours, price
- "Enrol" CTA (sticky bottom bar on mobile)
- Curriculum accordion: Module → Topics list (topics show type icon + estimated minutes)
- Module 1 preview badge ("Free preview")
- Reviews/social proof section (Phase 2 placeholder)

/roles/[slug]/learn (learning flow — only after enrollment)
- Full-screen content view optimised for reading

- Fixed top bar: back button, module title, progress dots
- Content rendered from JSON (Portable Text renderer for Sanity content)
- Fixed bottom bar: "Mark as Complete" CTA
- TTS floating action button: tap to start/stop audio
- TTS player: appears as bottom sheet when active (play/pause, speed, progress)
- Swipe left/right to navigate topics (after marking complete)
- Content types render differently:
  - text: clean readable prose with proper typographic spacing
  - guide: numbered step list with step numbers highlighted in brand blue
  - case_study: dialogue-style layout, "Scenario" header, "Resolution" section
  - workflow: vertical step-by-step with connector lines between steps

/tests/[test_id] (test taking)
- Pre-test screen: title, question count, time limit, pass mark
- "Start Test" CTA
- During test:
  - One question per screen (full-screen, swipe disabled)
  - Question counter: "3 of 12"
  - MCQ options as tappable cards (highlight on select, no auto-advance)
  - "Next" button (disabled until option selected)
  - Timer countdown in top right (if time_limit set)
  - Progress bar across top
- Results screen:
  - Score ring animation (count up to final score)
  - Pass: confetti burst + "Congratulations!" + score + XP earned + badges earned
  - Fail: "Keep going!" + score + "Retry in {N} hours" or "Try Again" if no cooldown
  - If final exam passed: special celebration screen + certificate preview + share CTAs

/certificates
- Grid of earned certificates (certificate card with PNG preview)
- Tap → certificate detail with share options:
  WhatsApp button (opens wa.me with pre-written text)
  Copy link button
  Download button (downloads PNG)

/roadmap
- Visual career path: current role(s) → adjacent roles → next level roles
- Completed roles: filled circle with certificate badge
- Current role: pulsing ring
- Next role: outlined circle with "Enrol" CTA
- Render as a vertical swimlane on mobile

/jobhub (for subscribed learners)
- If not subscribed: paywall screen with subscription pricing + "Subscribe" CTA → payment flow

```
- If subscribed:
  - Profile settings (preferred roles, location, availability)
  - Matches list: role, employer (or "Anonymous"), location, pay range, status badge
  - Accept / Decline actions on pending matches
  - Match notification history

PWA CONFIGURATION:
- manifest.json: name="InTrainin", short_name="InTrainin", theme_color="#1A56DB", background_color="#FFFFFF", display="standalone", start_url="/dashboard"
- Service worker (via next-pwa): cache static assets; offline page for network failures
- Install prompt: show "Add to Home Screen" banner after 3rd visit if not installed

GLOBAL UI REQUIREMENTS:
- Bottom navigation bar (mobile): Home, Explore, My Learning, Profile
- Top bar only on pages that need it (not learning flow — full screen there)
- All loading states use skeleton components (not spinners)
- All error states have a retry action
- Toast notifications for: course enrolled, topic complete, test passed, badge earned, certificate ready
- All monetary amounts formatted as "₦2,000" (Nigerian naira, comma-separated)
```

Acceptance Criteria

- OTP flow completes end-to-end on mobile viewport (375px width)
- Role catalogue loads under 2 seconds (with skeleton loading visible)
- TTS bottom sheet plays audio and persists across topic navigation
- Test taking: timer counts down; submission blocked until all questions answered
- Certificate share: WhatsApp deeplink opens with correct pre-written text
- Roadmap renders correctly with locked/unlocked states
- PWA installable: manifest valid, offline page works
- All tap targets ⩾ 44px

Prompt 10 — Notifications System

Context

Three channels: in-app (DB-backed), push (FCM), and SMS (Termii). Smart re-engagement cadence. Preference controls. This runs as a background service.

Pre-conditions

- All previous prompts complete
- Firebase project with FCM enabled

The Prompt

```
Implement the full notifications system in apps/api/src/domains/notifications/.

NOTIFICATION SERVICE (notifications/service.ts):

Create a NotificationService class with these methods:

async send(userId: string, notification: NotificationPayload): Promise<void>
- NotificationPayload: { type, title, body, data?: object, channels: ('in_app' |
'push' | 'sms')[] }
- Fetch user's notification_prefs and fcm_token
- For each channel in payload.channels (respecting user prefs):
  - in_app: INSERT into notifications table
  - push: if user has fcm_token and push pref is true → send via FCM
  - sms: if user has phone and sms pref is true → send via Termii
- Never throw on notification failure — log errors but don't fail the parent
operation
- Log all sends to a notification_log table (user_id, channel, type, status,
sent_at)

FCM SEND (notifications/fcm.ts):
- Use firebase-admin SDK
- sendToDevice(fcm_token, { title, body, data }) → returns success/failure
- Handle expired/invalid token: if FCM returns UNREGISTERED or INVALID_ARGUMENT →
clear user's fcm_token in DB

TERMII SEND (notifications/termii.ts):
- POST to Termii /sms/send
- Sender ID: InTrainin
- Log all responses

NOTIFICATION TYPES TO IMPLEMENT:
Define all notification templates in notifications/templates.ts:

type | title | body template
course_enrolled | "Enrolled! 🎉" | "You're now enrolled in {role_title}. Let's
start learning!"
topic_completed | "Topic complete ✓" | "Great job completing '{topic_title}'. Keep
going!"
module_test_unlocked | "Test unlocked 🔓" | "You've completed all topics in
{module_title}. Take the module test now."
test_passed | "Test passed! 🏆" | "You scored {score}% on {test_title}.
```

{xp_awarded} XP earned."
test_failed | "Keep going 💪" | "You scored {score}% on {test_title}. You need {pass_mark}% to pass. Retry in {cooldown} hours."
certificate_issued | "Certificate ready! 🎓" | "You've earned your {role_title} certificate! Tap to view and share."
badge_earned | "Badge unlocked! 🏅" | "You earned the '{badge_name}' badge. Well done!"
streak_reminder | "Keep your streak 🔥" | "Your {n}-day streak is at risk! Complete a lesson today to keep it going."
job_match_new | "New job match 💼" | "A {role_title} opportunity in {city} matches your profile. Tap to view."
job_shortlisted | "You've been shortlisted! ⭐" | "{business_name} wants to interview you for {role_title}."
re_engagement_3d | "Miss you 👋" | "You haven't learned in 3 days. Your next topic is waiting."
re_engagement_7d | "Come back 📚" | "It's been a week! You're {pct}% through {role_title}. Keep going."

RE-ENGAGEMENT CRON JOB:
Create a scheduled job (run every hour via setInterval or a cron lib):

function checkReEngagement():
- Query users who:
  - Have at least one active enrollment (not completed)
  - streak_last_activity_date < NOW() - 3 days
  - Have not received a re_engagement notification in last 3 days (check notification_log)
- For 3-day inactive: send re_engagement_3d via push
- For 7-day inactive: send re_engagement_7d via push + SMS
- For 14-day inactive: send via push + SMS + email (weekly progress email)
- Cap: max 1 re-engagement per user per 3 days

DAILY STREAK CHECK:
Run at midnight (WAT = UTC+1):
- Find users whose streak_last_activity_date = YESTERDAY
- Send streak_reminder push to users with streak >= 3

FCM TOKEN REGISTRATION:
POST /notifications/fcm-token
- Auth required
- Body: { token: string }
- Update users.fcm_token
- Return { success: true }

GET /notifications/me
- Auth required
- Returns last 50 in-app notifications, unread count

```
- Marks all as read on fetch (UPDATE notifications SET is_read = true WHERE
user_id = ...)

POST /notifications/preferences
- Auth required
- Body: { push?: boolean, sms?: boolean, email?: boolean }
- Update users.notification_prefs

Add notification_log table:
CREATE TABLE notification_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  channel VARCHAR(20) NOT NULL,
  type VARCHAR(50) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'sent',
  error_message TEXT,
  sent_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Acceptance Criteria

- Notification failures don't propagate to parent operations (fire and forget)
- Re-engagement cron runs correctly; doesn't double-send to same user
- FCM token auto-clears on invalid token response
- User preferences respected: disable push → push not sent even if channel requested
- Notification log correctly records all send attempts including failures
- In-app notification count shows in web UI nav bar

Prompt 11 — Sanity CMS Schema

Context

All curriculum content lives in Sanity. Non-dev content editors update topics, guides, case studies, and workflows without touching code. This prompt sets up the full schema.

Pre-conditions

- Prompt 0 complete (packages/cms exists)
- Sanity project created

The Prompt

```
Create the complete Sanity v3 schema in packages/cms/schemas/.
```

DOCUMENT TYPES:

1. role (mirrors DB roles table — source of truth for content)
Fields: title (string, required), slug (slug), category (string, options: all 7 categories), description (text), modules (array of references to module documents)

2. module
Fields: title (string, required), role (reference to role), order_index (number), topics (array of references to topic documents)

3. topic
Fields:
- title (string, required)
- module (reference to module)
- content_type (string, options: text | guide | case_study | workflow)
- order_index (number)
- estimated_minutes (number)
- content (Portable Text — see content rules below)

PORTABLE TEXT CONTENT RULES:
Configure the content field to support:
- Standard blocks: normal, h3, h4, blockquote
- Marks: strong, em, underline
- Custom block types:
  a. stepItem: for guide steps — fields: step_number (number), instruction (text), tip (text, optional)
  b. scenarioBlock: for case studies — fields: scenario_text (text), character (string: 'customer' | 'worker'), is_resolution (boolean)
  c. workflowStep: for workflows — fields: step_number (number), title (string), description (text), next_step_label (string, optional)
  d. highlightBox: informational callout — fields: variant ('info' | 'warning' | 'tip'), content (text)

4. test (document type for test questions — synced to DB)
Fields:
- title (string)
- role (reference to role)
- module (reference to module, optional)
- test_type (string: module | final)
- pass_mark_pct (number, default 70)
- time_limit_minutes (number, optional)
- questions (array of question objects):
  Each question:
  - question_id (string, auto-generated UUID)
  - question_text (text, required)
  - question_type (string: mcq | scenario)

```
  - options (array of strings, 4 items)
  - correct_answer (string — must match one of options)
  - explanation (text, optional — shown after test)

STUDIO CONFIGURATION:
- Desk structure: group documents as Role Library → [by category] → role → modules
→ topics
- Preview: show role title + category in list view
- Validation: require at least 2 topics per module, at least 4 options per
question, correct_answer must match an option
- API version: '2024-01-01'

SYNC WEBHOOK:
In apps/api, add a Sanity webhook endpoint:
POST /cms/sync (protected by a shared SANITY_WEBHOOK_SECRET header)
- Triggered by Sanity on document publish events
- For topic documents: fetch content from Sanity by _id, update
topics.content_body in DB, update topics.sanity_id
- For test documents: sync questions array to tests table
- Log sync events

Create a one-time sync script in packages/cms/scripts/initialSync.ts that:
- Fetches all published role/module/topic/test documents from Sanity
- Upserts them into the DB
- Use this to initialise DB from Sanity content on first deploy
```

Acceptance Criteria

- Sanity Studio loads and all document types visible
- Custom block types render in Studio preview
- Validation errors show for incomplete questions
- Webhook updates DB content_body within 5 seconds of Sanity publish
- Initial sync script populates DB from Sanity correctly

Prompt 12 — End-to-End Testing & Deployment

Context

Integration tests for critical paths and deployment configuration. This is the final gate before going live.

Pre-conditions

- All previous prompts complete

The Prompt

```
Write integration tests for all critical user paths and configure deployment.

TESTING (use Vitest + Supertest for API, Playwright for E2E):

API Integration Tests (apps/api/src/__tests__/):

1. auth.test.ts
   - Phone OTP: request OTP → verify OTP → get JWT → use JWT → refresh token →
logout
   - Invalid OTP rejected
   - Expired OTP rejected
   - Rate limiting: 4th OTP request within 1 hour is blocked

2. learning.test.ts
   - Enrol in role → get progress (0%) → complete topics in order → blocked if out
of order → module test unlocks
   - XP awards: confirm XP increases on topic complete and test pass
   - Streak: complete topic day 1, day 2 increments, day 4 resets

3. assessment.test.ts
   - Module test blocked before all topics done
   - Final exam blocked before all module tests passed
   - Correct scoring: submit all correct → 100%, submit all wrong → 0%
   - Cooldown enforced on failed attempt
   - Final exam pass → certificate generated
   - Badge awarded on 90%+ score (idempotent: awarded only once)

4. payment.test.ts
   - Webhook with invalid HMAC rejected (400)
   - Webhook charge.success creates enrollment (test with mock Paystack response)
   - Tampered amount rejected
   - Duplicate webhook reference ignored (idempotent)

5. jobhub.test.ts
   - Unsubscribed user cannot see matches
   - Matching algorithm: certified + same city scores highest
   - Shortlisting reveals full contact info

E2E Tests (apps/web/e2e/ using Playwright):

1. learner_journey.spec.ts
   Complete flow: sign up (mock OTP) → onboarding → enrol in Cashier course (mock
payment) → complete 3 topics → take module test → pass → see certificate
```

```
2. business_journey.spec.ts
   Register business → subscribe (mock payment) → add team member via CSV → view
team progress dashboard

3. certificate_verify.spec.ts
   Generate certificate → visit /verify/{code} → verify valid certificate shows →
visit invalid code → shows invalid state

DEPLOYMENT:

Docker configuration:
- apps/api/Dockerfile: Node.js 20 Alpine, multi-stage build, non-root user
- docker-compose.yml for local development: api, postgres (local), redis (for rate
limiting)

Environment-specific configs:
- .env.production.example with all vars needed for deployment
- Document which vars are secret (never in git) vs which can be committed

Deployment targets:
- apps/api → Railway (Dockerfile-based deploy)
- apps/web → Vercel (Next.js native)
- packages/db → Supabase (managed)

Create a DEPLOYMENT.md with:
1. First-time setup steps (Supabase project, Sanity project, Paystack webhooks,
Firebase, Termii, Cloudinary accounts)
2. Environment variable checklist
3. Database migration steps
4. Initial Sanity content sync command
5. How to run initial seed data
6. Paystack webhook URL configuration (must point to /payments/webhook)
7. FCM configuration for PWA (VAPID keys)
8. Health check endpoint: GET /health → { status: 'ok', version, db: 'connected' }

CI/CD (GitHub Actions):
- .github/workflows/ci.yml: on PR → lint, typecheck, run unit tests
- .github/workflows/deploy.yml: on merge to main → build, run tests, deploy api to
Railway + trigger Vercel deploy
```

Acceptance Criteria

- All integration tests pass
- E2E learner journey completes in < 60 seconds (headless)
- Docker build succeeds and API starts cleanly
- Health check endpoint returns 200 with db connected

- DEPLOYMENT.md covers all steps a new developer needs to go from zero to running production environment
- CI pipeline runs clean on a fresh branch

Quick Reference: API Route Summary

Domain	Method	Path	Auth	Notes
Auth	POST	/auth/otp/request	None	Rate limited
Auth	POST	/auth/otp/verify	None	Returns JWT
Auth	POST	/auth/email/register	None	Business only
Auth	POST	/auth/email/login	None	
Auth	POST	/auth/refresh	None	Refresh token in body
Auth	POST	/auth/logout	JWT	
Auth	PATCH	/auth/profile	JWT	
Learning	GET	/roles	None	Cached 15min
Learning	GET	/roles/:slug	None	
Learning	POST	/enrollments	JWT	Paystack verify
Learning	GET	/enrollments/me	JWT	
Learning	GET	/learning/:enrollment_id/progress	JWT	
Learning	POST	/learning/topics/:id/complete	JWT	Awards XP
Learning	GET	/learning/topics/:id/content	JWT	
Learning	GET	/learning/topics/:id/audio	JWT	TTS cached
Assessment	GET	/tests/:id	JWT	Access check
Assessment	GET	/tests/:id/questions	JWT	No answers
Assessment	POST	/tests/:id/submit	JWT	Triggers cert
Assessment	GET	/tests/:id/attempts	JWT	

Domain	Method	Path	Auth	Notes
Certificates	GET	/certificates/me	JWT	
Certificates	GET	/verify/:code	None	Rate limited
Payments	POST	/payments/course/initiate	JWT	
Payments	POST	/payments/jobhub/initiate	JWT	
Payments	POST	/payments/hire-request/initiate	JWT Business	
Payments	POST	/payments/enterprise/initiate	JWT Business	
Payments	POST	/payments/webhook	None	HMAC verified
Payments	GET	/payments/status/:ref	JWT	
Job Hub	POST	/jobhub/profile	JWT	
Job Hub	GET	/jobhub/matches	JWT Subscribed	
Job Hub	PATCH	/jobhub/matches/:id	JWT	
Job Hub	POST	/jobhub/hire-requests	JWT Business	
Job Hub	GET	/jobhub/hire-requests/:id/candidates	JWT Business	
Job Hub	POST	/jobhub/hire-requests/:id/shortlist	JWT Business	
Business	GET	/business/me	JWT Business	
Business	POST	/business/members	JWT Business	
Business	POST	/business/members/bulk	JWT Business	CSV upload
Business	GET	/business/members	JWT Business	
Business	GET	/business/analytics	JWT Business	
Notifications	POST	/notifications/fcm-token	JWT	
Notifications	GET	/notifications/me	JWT	Marks read
Notifications	POST	/notifications/preferences	JWT	
CMS	POST	/cms/sync	Webhook secret	Sanity trigger
System	GET	/health	None	Uptime check

Critical Rules for Claude Code Sessions

1. **Never trust client-side payment confirmation.** Enrollment is only created from the Paystack webhook. Period.
2. **Always enforce linear progression server-side.** The UI prevents out-of-order navigation, but the API must also validate it.
3. **Certificates are permanent.** Once issued, `is_revoked` must only be set by an admin. Never auto-revoke.
4. **RLS is not optional.** Every query that touches user data must have a corresponding RLS policy. Verify with the anon key, not service role.
5. **Rate limit everything sensitive.** OTP requests, certificate verification lookups, and webhook endpoints all need rate limits.
6. **TTS and certificate generation are async-tolerant.** If they fail, they should not fail the parent operation. Return a success response and retry in background.
7. **No PII in URLs or logs.** Use UUIDs everywhere. Never log phone numbers, emails, or names.
8. **Paystack HMAC on every webhook.** Verify before any processing. Log and discard tampered requests.
9. **Full name only revealed on shortlist.** Job Hub candidates show first name only until a business shortlists them.
10. **Content body is cached in DB.** After first Sanity fetch, store in `topics.content_body`. Only re-fetch on Sanity webhook invalidation.